

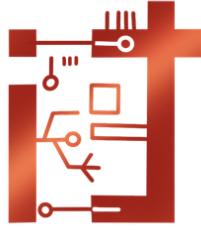
Project Report

Formal Test Design & Automation

This project implements **Model-Driven Test Design (MDTD)** and **Input Space Partitioning (ISP)** to verify a discount calculation system. It focuses on deriving **Pair-Wise Coverage (PWC)** test cases, which are automated using JUnit 5 parameterized tests to validate functional logic and handle infeasible input constraints.

	Student Number	Section
Fatima Eid	2239352	1
Areej Al-Masri	2334600	1
Liyan Ewies	2336703	1

Instructor: Dr. Bashar Shboul

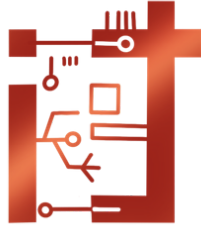


MDTD & ISP Design

Characteristics and Blocks

To design the test suite, the input space was partitioned into three disjoint and complete blocks.

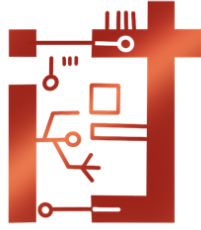
Characteristic	Definition	Blocks
C_1 : CustomerType	The category of the customer.	b_1 : "NEW", b_2 : "REGULAR", b_3 : "PREMIUM"
C_2 : totalOrders	Orders made in the last year.	b_1 : < 10 , b_2 : ≥ 10
C_3 : isSubscribed	Newsletter subscription status.	b_1 : $< \text{true}$, b_2 : false



Pair-Wise Coverage (PWC) Combinations

This section presents the test suite derived to satisfy **Pair-Wise Coverage (PWC)**. The goal of this combinatorial approach is to ensure that every possible pair of characteristic values is tested at least once, which effectively identifies faults caused by parameter interactions while maintaining a manageable test suite size.

Test Case	customerType	totalOrders	isSubscribed	Expected Discount
TC1	"NEW"	5	TRUE	7.0% (5% base + 2% sub)
TC2	"NEW"	2	FALSE	5.0% (5% base)
TC3	"REGULAR"	5	FALSE	8.0% (5% base + 3% type)
TC4	"REGULAR"	15	TRUE	15.0% (5% base + 3% type + 5% loyalty + 2% sub)
TC5	"PREMIUM"	5	TRUE	12.0% (5% base + 5% type + 2% sub)
TC6	"PREMIUM"	15	FALSE	15.0% (5% base + 5% type + 5% loyalty)



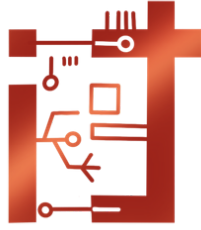
Infeasibility Justification

One combination was identified as mathematically and logically infeasible within the system rules.

Infeasible Pair: `customerType = "NEW"` AND `totalOrdersInLastYear 10$`.

Justification: According to Functional Requirement #5, a customer categorized as "NEW" cannot have 10 or more total orders from the previous year.

Implementation: This is handled by throwing an `IllegalArgumentException` in the code, verified by the `testInfeasibleCombination()` unit test.

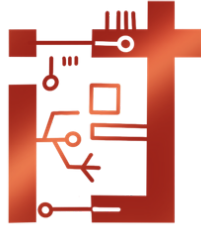


Limitations of Pair-Wise Testing

While **Pair-Wise Coverage (PWC)** is efficient for identifying bugs caused by the interaction of two parameters, it is not exhaustive.

The primary limitation is that PWC does not guarantee a bug-free system because it may miss 3-way (or higher) interaction faults. A bug might only occur when a specific combination of three variables—such as **"PREMIUM", $\geq 10\$$ orders"**, AND **"Subscribed"**—triggers a unique logic path.

Since PWC only ensures every pair is tested, this specific triplet might never be executed unless a 3-way combinatorial or exhaustive testing strategy is used.



Individual and Team Work: Contribution Matrix

Student Name	Core Contributions
Fatima Eid	MDTD & ISP Modeling: Identified characteristics and defined disjoint/complete blocks. Performed the infeasibility analysis and provided the logical justification for the "NEW" customer constraint.
Areej Al-Masri	System Implementation & Automation: Developed the functional logic for DiscountCalculator.java and implemented the automated @ParameterizedTest suite in DiscountCalculatorTest.java.
Liyan Ewies	Report Documentation & Analysis: Authored and formatted the final ProjectReport.pdf. Conducted the technical analysis regarding the "Limitations of Pair-Wise Testing" compared to exhaustive testing.